lecture 12
hash tables

Summary
midterm 1: sorting
midterm 2: searching, dynamic search, balanced BST

	balanced BST	direct addressing table
operations	insert, delete, search, min, max, read in sorted order	insert, search, delete
time	$\simeq O(\log(n))$	$\simeq O(1)$

Direct Addressing Table

- new data structure
- array structure. each element is named a bucket

insert(v, k)
- just use the value v as the index and set it as the value
- assume there is an empty value

search(v, k)
- check index v if there is a value there

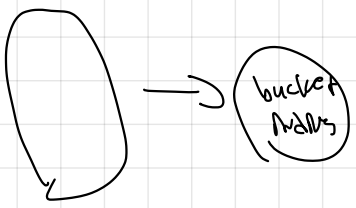
delete(v, k)
- put empty value at index v

Time: $O(1)$
Space: $O(m)$ with m buckets

cons: the number of buckets is fixed
empty slots need to be minimized
need to treat values as indices
- does not work for non integer types

Best for a small fixed domain of values

Hash Function
- Suppose I have arbitrary numbers, but need to map it to a set of finite bucket indices



$\mathbb{N} \rightarrow [0, m)$

example: $h(x) = x \bmod m$

Hash Table
- direct addressing table that uses a hash function to generate the index of the bucket

insert 13, 4, 100

$h(13) = 3$
$h(4) = 4$
$h(100) = 0$

100	_	_	13	4	_	_	_	_	_
0	1	2	3	4	5	6	7	8	9

insert 20

$h(20) = 0$ Collision

100	_	_	13	4	_	_	_	_	_
0	1	2	3	4	5	6	7	8	9

Separate chaining
- a list at each bucket

index 0 is already taken, we need a dynamic array here
or linked list
↑ not preferred since we can't cache it

assume we never insert duplicates

```
def insert(buckets, v):
    if search(buckets, v):
        return
    (maybe rehash)

    buckets[hash(v)].prepend(v)
```

] multiset does not have this line

```
def search(buckets, v):
    for node in buckets[hash(v)]:
        if node.value == v:
            return true
    return false

def delete(buckets, v):
    for node in buckets[hash(v)]:
        if node.value == v:
            delete node from list
    (maybe rehash)
```

---

uniformly random hash function h
n items with m buckets in a hash table

$E[\#\text{ of collisions}] = \binom{n}{2}\frac{1}{m}$ at least n-m collisions

Probability a pair of numbers collide in the hash table
use indicator variables:

two vars: v and w

$$P(h(v)=h(w)) = m\left(\frac{1}{m}\cdot\frac{1}{m}\right) = \frac{1}{m}$$

(m values; same value)

Need to check each pair

$$\sum_{(v,w)} \frac{1}{m}\cdot 1 = \binom{n}{2}\frac{1}{m}$$

collision: pair of elements that collide

---

we want all buckets to not be full, so searching is still O(1)

we don't know the input beforehand

random is a good approximation for distributing evenly

expected number of items per bucket

Expectation across all items (0,...n-1), did item i land in bucket b

$$\sum_{i=1}^{n} Pr(h(i)=b) = \sum_{i=1}^{n}\frac{1}{m} = \frac{n}{m}$$

we want every bucket to have 1 value, so space is not wasted and allows O(1) search

we want $n \geq m$

Rehash — don't need to worry about it

pick an initial size for the hash table
Once that is reached, double the size of buckets

How many people must be in a party before at least 2 people with the same birthday

$$365\left(\frac{1}{365}\right)\left(\frac{1}{365}\right)$$

pick uniformly random hash function
perform n operations

initially:
- empty hash table with n buckets
- separate-chaining collision handling

Determine worst case sequence of n operations and expected time

if we don't know the hash function, each operation is expected $O(1)$

the hash function will convert the value to a random value
- worst case input becomes average case input

Total time: n operations · $O(1)$
$O(n)$

---

hash func:
$h(x) = x \bmod n$

worst case sequence of n operations and the expected time

insert x } n times — insert to the same bucket
- equally likely for operation i

$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$

---

linear probing

insert:
if there is already a value at the bucket we want to insert to, go to the next bucket until we find an empty bucket

try until you circle around

search:
keep searching with the same advancing logic

delete:
- have a deleted value, so searches continue